

THE CLAUDE BEAST | EPISODE 01

System Prompt

CLAUDE.md & AGENTS.md

*How to give Claude persistent memory, project context,
and cross-tool instructions that survive every session.*

The Problem: Claude Starts Every Session Blank

Why persistent context changes everything



Without CLAUDE.md

- You repeat architecture decisions every session
- Code style preferences get forgotten
- Build commands must be re-explained
- Module relationships are not visible to Claude
- Domain terminology causes confusion



With CLAUDE.md

- Claude knows your stack from line 1
- Coding conventions enforced automatically
- Build/test commands always available
- Architecture context persists across sessions
- Domain language is pre-loaded and correct

What is CLAUDE.md?

The file that gives Claude persistent project memory

A markdown file that gives Claude Code persistent, project-specific context. It is automatically loaded into Claude's system prompt at the start of every session.

- **The only file** — loaded by default into every conversation with the agent
- Lives in your repo root as CLAUDE.md or .claude/CLAUDE.md
- **Claude reads it automatically** — no prompting required
- Becomes part of Claude's system prompt (injected in <system-reminder> tags)
- Eliminates repeating project structure, conventions, and workflows
- Committed to source control so your entire team shares the same context

The 6-Level Memory Hierarchy

More specific memory takes precedence over broader memory.

1	Managed Policy	/Library/Application Support/ClaudeCode/CLAUDE.md	Org-wide rules (IT/DevOps managed)
2	User Memory	~/.claude/CLAUDE.md	Personal prefs across all projects
3	Project Memory	./CLAUDE.md or ~/.claude/CLAUDE.md	Team-shared project instructions
4	Project Rules	~/.claude/rules/*.md	Modular, path-specific rules
5	Local Memory	./CLAUDE.local.md	Personal project-specific prefs
6	Auto Memory	~/.claude/projects/<proj>/memory/	Claude's own notes (first 200 lines)

Key Sections to Include in Your CLAUDE.md

The essential building blocks of effective project context

Project Overview

Framework, tech, one-liner description

Workflows

Step-by-step for common tasks (e.g., new API endpoint)

Tech Stack

Framework, DB, auth, API versioning

Do's and Don'ts

Critical rules: what to never modify, never use

Common Commands

dev, test, lint, migrate -- per-file when possible

Domain Terminology

Business terms the model might misunderstand

Code Conventions

Strict mode, named exports, test co-location

Gotchas

NEVER commit .env, auto-generated files, signatures

Also consider: MCP Server instructions, import paths (@path), and custom slash commands.

AGENTS.md: The Cross-Tool Standard

README.md is for humans. AGENTS.md is for AI agents.

An open, tool-agnostic standard for providing plain-text context to any AI agent.

Supported by:

Cursor | Zed | Codex (OpenAI) | Jules (Google) | GitHub Copilot | Gemini CLI | Windsurf |
Aider | Warp | VS Code

The Symlink Pattern (Interoperability)



Option A: Reference

CLAUDE.md contains:
"Strictly follow the rules in ./AGENTS.md"



Option B: Symlink

`ln -s AGENTS.md CLAUDE.md`



Option C: Migrate

Rename CLAUDE.md to AGENTS.md, then
symlink back for compatibility

Best Practices: Less Is More

Your instruction budget is finite -- spend it wisely

- **Keep it concise** — aim for under 300 lines (some recommend under 60 for root file)
- As instruction count rises, instruction-following quality drops uniformly
- LLMs bias toward instructions at the beginning and end of the prompt
- **Use progressive disclosure** — keep detailed docs in separate files, reference with @imports
- **Never use CLAUDE.md as a linter** — use ESLint, Prettier, Biome + hooks instead
- **Treat it as a living document** — add instructions as you work, not just upfront
- **Use emphasis sparingly** — IMPORTANT, NEVER, YOU MUST work, but lose power if overused
- Commit to source control; use CLAUDE.local.md for personal preferences
- Never include secrets, API keys, or credentials

Getting Started: The /init Command

Bootstrap your CLAUDE.md in seconds



terminal

```
$ cd your-project  
$ claude  
> /init
```

What /init does:

- Examines your codebase: package files, docs, config, code structure
- Generates a starter CLAUDE.md with build commands and conventions
- Can also improve an existing CLAUDE.md



Treat /init as a starting point, not the final product

- The generated file includes obvious details; the real value comes from iterating
- **Delete from the generated output what is not needed** — easier than writing from scratch
- Add instructions as you encounter friction with Claude over time
- Use /memory to review and organize Claude's auto-generated learnings
- Periodically ask Claude to review CLAUDE.md for redundancies and outdated info

Modular Rules & Advanced Features

Scale your instructions without bloating the root file

Path-Specific Rules (.claude/rules/)

- Organize instructions into multiple .md files
- Use YAML frontmatter to scope rules to file globs
- Example: paths: ["src/api/**/*.ts"] loads only for API files
- Supports brace expansion: *.ts,tsx}
- Symlink shared rules across repos

Advanced Features

- **@path imports** — reference external files (max 5 hops deep)
- **Auto Memory** — Claude writes its own notes as it works
- **Custom Slash Commands** — reusable prompts in .claude/commands/
- **Subagents** — specialized AI with custom system prompts
- **Recursive lookup** — Claude reads CLAUDE.md up the directory tree



Context Window Impact

Claude Code's system prompt has ~50 built-in instructions. Frontier LLMs follow ~150-200 instructions consistently. Every line in CLAUDE.md competes for attention with the actual work.

Key Takeaways

Your action items for mastering Claude's memory

- **CLAUDE.md** is the single highest-leverage file — loaded before every conversation
- **Six-level hierarchy** — provides granular control from org-wide to auto memory
- **Less is more** — keep instructions concise, specific, and universally applicable
- **Progressive disclosure** — use @imports and .claude/rules/ instead of one massive file
- **AGENTS.md** — the tool-agnostic standard; symlink for cross-tool compatibility
- **Living document** — evolve through actual usage friction, not upfront planning
- **Never include sensitive information** — no API keys, credentials, or secrets



```
Next step:  cd your-project && claude && /init
```